

OAuth

OAuth (short for **open authorization**^{[1][2]}) is an open standard for access delegation, commonly used as a way for internet users to grant websites or applications access to their information on other websites but without giving them the passwords.^{[3][4]} This mechanism is used by companies such as Amazon,^[5] Google, Meta Platforms, Microsoft, and Twitter to permit users to share information about their accounts with third-party applications or websites.

Generally, the OAuth protocol provides a way for resource owners to provide a client application with secure delegated access to server resources. It specifies a process for resource owners to authorize third-party access to their server resources without providing credentials. Designed specifically to work with Hypertext Transfer Protocol (HTTP), OAuth essentially allows access tokens to be issued to third-party clients by an authorization server, with the approval of the resource owner. The third party then uses the access token to access the protected resources hosted by the resource server.^[2]



Unofficial logo designed by Chris Messina

Latest version	2.0
Organization	<u>Internet Engineering Task Force</u>
Website	<u>IETF OAuth Datatracker</u> (https://datatracker.ietf.org/wg/oauth/about/)

History

OAuth began in November 2006 when Blaine Cook was developing an OpenID implementation for Twitter. Meanwhile, Ma.gnolia needed a solution to allow its members with OpenIDs to authorize Mac OS X Dashboard widgets to access their service. Cook, Chris Messina and Larry Halff from Magnolia met with David Recordon to discuss using OpenID with the Twitter and Magnolia APIs to delegate authentication. They concluded that there were no open standards for API access delegation.^[6]

The OAuth discussion group was created in April 2007, for a small group of implementers to write the draft proposal for an open protocol. DeWitt Clinton from Google learned of the OAuth project, and expressed his interest in supporting the effort. In July 2007, the team drafted an initial specification. Eran Hammer joined and coordinated the many OAuth contributions creating a more formal specification. On 3 October 2007, the OAuth Core 1.0 final draft was released.^[6]



A hypothetical authorization flow where login information is shared with a third-party application. This poses many security risks which can be prevented by the use of OAuth authorization flows.

At the 73rd Internet Engineering Task Force (IETF) meeting in Minneapolis in November 2008, an OAuth BoF was held to discuss bringing the protocol into the IETF for further standardization work. The event was well attended and there was wide support for formally chartering an OAuth working group within the IETF.

The OAuth 1.0 protocol was published as RFC 5849, an informational Request for Comments, in April 2010. Since 31 August 2010, all third party Twitter applications have been required to use OAuth.^[7]

The OAuth 2.0 framework was published considering additional use cases and extensibility requirements gathered from the wider IETF community. Albeit being built on the OAuth 1.0 deployment experience, OAuth 2.0 is not backwards compatible with OAuth 1.0. OAuth 2.0 was published as RFC 6749 and the Bearer Token Usage specification as RFC 6750, both standards track Requests for Comments, in October 2012.^{[2][8]}

As of November 2024, the OAuth 2.1 Authorization Framework draft is a work in progress. It consolidates the functionality in RFCs OAuth 2.0, OAuth 2.0 for Native Apps, Proof Key for Code Exchange, OAuth 2.0 for Browser-Based Apps, OAuth Security Best Current, and Bearer Token Usage.^[9]

Security issues

OAuth 1.0

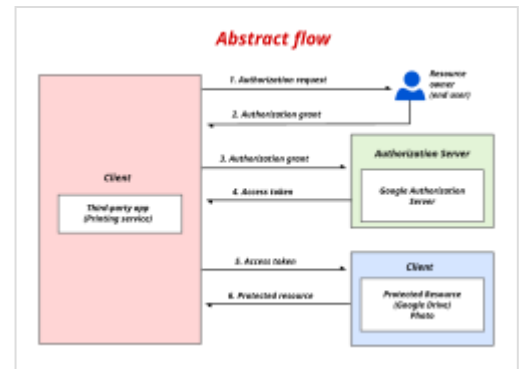
On 23 April 2009, a session fixation security flaw in the 1.0 protocol was announced. It affects the OAuth authorization flow (also known as "3-legged OAuth") in OAuth Core 1.0 Section 6.^[10] Version 1.0a of the OAuth Core protocol was issued to address this issue.^[11]

OAuth 2.0

In January 2013, the Internet Engineering Task Force published a threat model for OAuth 2.0.^[12] Among the threats outlined is one called "Open Redirector"; in early 2014, a variant of this was described under the name "Covert Redirect" by Wang Jing.^{[13][14][15][16]}

OAuth 2.0 has been analyzed using formal web protocol analysis. This analysis revealed that in setups with multiple authorization servers, one of which is behaving maliciously, clients can become confused about the authorization server to use and may forward secrets to the malicious authorization server (AS Mix-Up Attack).^[17] This prompted the creation of a new best current practice internet draft that sets out to define a new security standard for OAuth 2.0.^[18] Assuming a fix against the AS Mix-Up Attack in place, the security of OAuth 2.0 has been proven under strong attacker models using formal analysis.^[17]

One implementation of OAuth 2.0 with numerous security flaws has been exposed.^[19]



A high-level overview of OAuth 2.0 flow. The resource owner credentials are used only on the authorization server, but not on the client (e.g. the third-party app).

In April and May 2017, about one million users of [Gmail](#) (less than 0.1% of users as of May 2017) were targeted by an OAuth-based phishing attack, receiving an email purporting to be from a colleague, employer or friend wanting to share a document on Google Docs.^[20] Those who clicked on the link within the email were directed to sign in and allow a potentially malicious third-party program called "Google Apps" to access their "email account, contacts and online documents".^[20] Within "approximately one hour",^[20] the phishing attack was stopped by Google, who advised those who had given "Google Apps" access to their email to revoke such access and change their passwords.

In the draft of OAuth 2.1 the use of the PKCE ([RFC 7636](#)) extension for native apps has been recommended to all kinds of OAuth clients, including web applications and other confidential clients in order to prevent malicious browser extensions from performing OAuth 2.0 code injection attacks.^[9]

Types

OAuth framework specifies several grant types for different use cases. Some of the most common OAuth grant types are:^[21]

- Authorization Code
- PKCE
- Client Credentials
- Device Code
- Refresh Token
- Resource Owner Password Credentials (ROPC)

Uses

Facebook's [Graph API](#) only supports OAuth 2.0.^[22] [Google](#) supports OAuth 2.0 as the recommended authorization mechanism for all of its APIs.^[23] [Microsoft](#) also supports OAuth 2.0 for various APIs and its Azure Active Directory service,^[24] which is used to secure many Microsoft and third party APIs.

OAuth can be used as an authorizing mechanism to access secured [RSS/Atom](#) feeds. Access to RSS/ATOM feeds that require authentication has always been an issue. For example, an RSS feed from a secured [Google Site](#) could not have been accessed using [Google Reader](#). Instead, three-legged OAuth would have been used to authorize that RSS client to access the feed from the Google Site.

Free software client implementations of the OAuth2 protocol such as the [LibreOffice OAuth2OOo](#) (<http://prvchr.github.io/OAuth2OOo/>) extension allows access to remote resources (ie: via the [Google API](#) or the [Microsoft Graph API](#) and OAuth 2.0) and possibly even with the [LibreOffice Basic](#) language. This makes it very easy to write and use HTTP requests supporting the OAuth 2.0 protocol in [LibreOffice macros](#).

OAuth and other standards

OAuth is a service that is complementary to and distinct from OpenID. OAuth is unrelated to OATH, which is a reference architecture for authentication, not a standard for authorization. However, OAuth is directly related to OpenID Connect (OIDC), since OIDC is an authentication layer built on top of OAuth 2.0. OAuth is also unrelated to XACML, which is an authorization policy standard. OAuth can be used in conjunction with XACML, where OAuth is used for ownership consent and access delegation whereas XACML is used to define the authorization policies (e.g., managers can view documents in their region).

OpenID vis-à-vis pseudo-authentication using OAuth

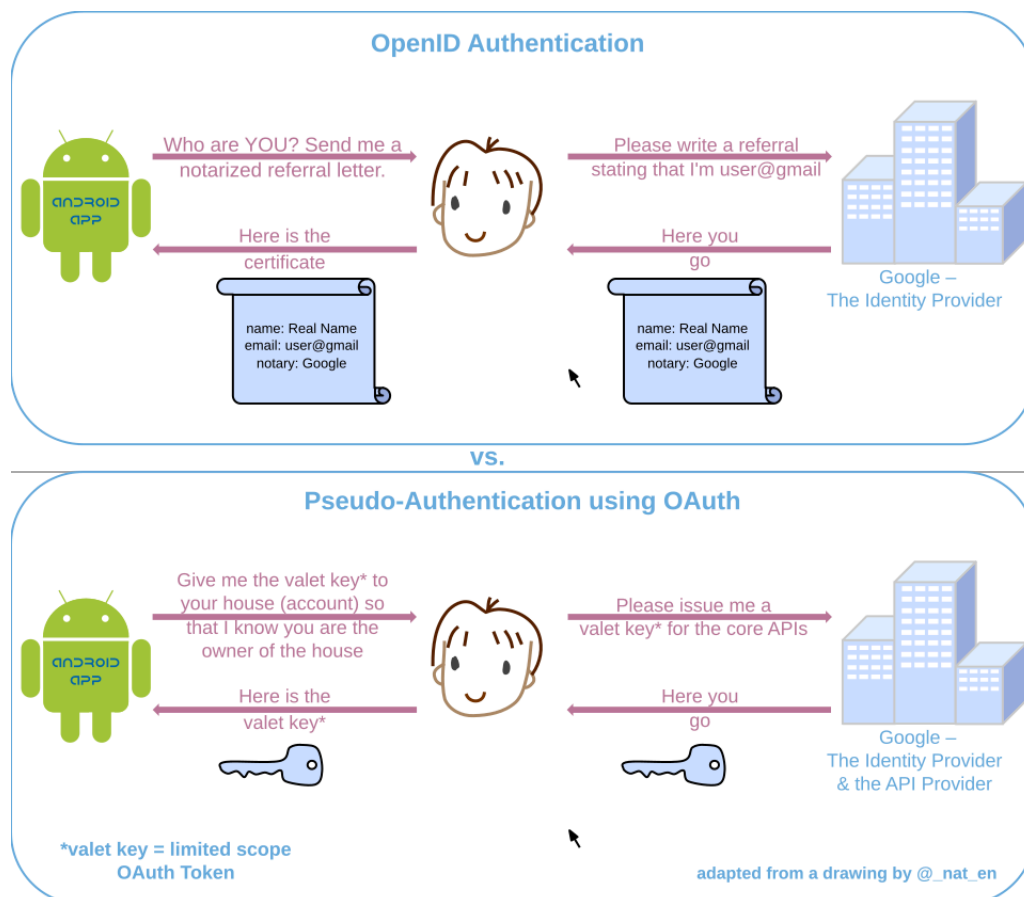
OAuth is an *authorization* protocol, rather than an *authentication* protocol. Using OAuth on its own as an authentication method may be referred to as pseudo-authentication.^[25] The following diagrams highlight the differences between using OpenID (specifically designed as an authentication protocol) and OAuth for authorization.

The communication flow in both processes is similar:

1. (Not pictured) The user requests a resource or site login from the application.
2. The site sees that the user is not authenticated. It formulates a request for the identity provider, encodes it, and sends it to the user as part of a redirect URL.
3. The user's browser makes a request to the redirect URL for the identity provider, including the application's request
4. If necessary, the identity provider authenticates the user (perhaps by asking them for their username and password)
5. Once the identity provider is satisfied that the user is sufficiently authenticated, it processes the application's request, formulates a response, and sends that back to the user along with a redirect URL back to the application.
6. The user's browser requests the redirect URL that goes back to the application, including the identity provider's response
7. The application decodes the identity provider's response, and carries on accordingly.
8. (OAuth only) The response includes an access token which the application can use to gain direct access to the identity provider's services on the user's behalf.

The crucial difference is that in the OpenID *authentication* use case, the response from the identity provider is an assertion of identity; while in the OAuth *authorization* use case, the identity provider is also an API provider, and the response from the identity provider is an access token that may grant the application ongoing access to some of the identity provider's APIs, on the user's behalf. The access token acts as a kind of "valet key" that the application can include with its requests to the identity provider, which prove that it has permission from the user to access those APIs.

Because the identity provider typically (but not always) authenticates the user as part of the process of granting an OAuth access token, it is tempting to view a successful OAuth access token request as an authentication method itself. However, because OAuth was not designed with this use case in mind, making this assumption can lead to major security flaws.^[26]



OAuth and XACML

XACML is a policy-based, attribute-based access control authorization framework. It provides:

- An access control architecture.
- A policy language with which to express a wide range of access control policies including policies that can use consents handled / defined via OAuth.
- A request / response scheme to send and receive authorization requests.

XACML and OAuth can be combined to deliver a more comprehensive approach to authorization. OAuth does not provide a policy language with which to define access control policies. XACML can be used for its policy language.

Where OAuth focuses on delegated access (I, the user, grant Twitter access to my Facebook wall), and identity-centric authorization, XACML takes an attribute-based approach which can consider attributes of the user, the action, the resource, and the context (who, what, where, when, how). With XACML it is possible to define policies such as

- Managers can view documents in their department
- Managers can edit documents they own in draft mode

XACML provides more fine-grained access control than OAuth does. OAuth is limited in granularity to the coarse functionality (the scopes) exposed by the target service. As a result, it often makes sense to combine OAuth and XACML together where OAuth will provide the delegated access use case and consent management and XACML will provide the authorization policies that work on the applications, processes, and data.

Lastly, XACML can work transparently across multiple stacks (APIs, web SSO, ESBs, home-grown apps, databases...). OAuth focuses exclusively on HTTP-based apps.

Controversy

Eran Hammer resigned from his role of lead author for the OAuth 2.0 project, withdrew from the IETF working group, and removed his name from the specification in July 2012. Hammer cited a conflict between web and enterprise cultures as his reason for leaving, noting that IETF is a community that is "all about enterprise use cases" and "not capable of simple". "What is now offered is a blueprint for an authorization protocol", he noted, "that is the enterprise way", providing a "whole new frontier to sell consulting services and integration solutions".^[27] In comparing OAuth 2.0 with OAuth 1.0, Hammer points out that it has become "more complex, less interoperable, less useful, more incomplete, and most importantly, less secure". He explains how architectural changes for 2.0 unbound tokens from clients, removed all signatures and cryptography at a protocol level and added expiring tokens (because tokens could not be revoked) while complicating the processing of authorization. Numerous items were left unspecified or unlimited in the specification because "as has been the nature of this working group, no issue is too small to get stuck on or leave open for each implementation to decide."^[27]

David Recordon later also removed his name from the specifications for unspecified reasons. Dick Hardt took over the editor role, and the framework was published in October 2012.^[2]

David Harris, author of the email client Pegasus Mail, has criticised OAuth 2.0 as "an absolute dog's breakfast", requiring developers to write custom modules specific to each service (Gmail, Microsoft Mail services, etc.), and to register specifically with them.^[28]

See also

- List of OAuth providers
- Data portability
- IndieAuth
- Mozilla Persona
- Security Assertion Markup Language
- User-Managed Access

References

1. "Open Authorization - Glossary | CSRC" (https://csrc.nist.gov/glossary/term/open_authorization). NIST Computer Security Resource Center. Archived (https://web.archive.org/web/20220921224928/https://csrc.nist.gov/glossary/term/open_authorization) from the original on 21 September 2022. Retrieved 23 July 2021.
2. Hardt, Dick (October 2012). Hardt, D (ed.). "RFC6749 - The OAuth 2.0 Authorization Framework" (<https://tools.ietf.org/html/rfc6749>). Internet Engineering Task Force. doi:10.17487/RFC6749 (<https://doi.org/10.17487%2FRFC6749>). Archived (<https://web.archive.org/web/20121015184712/http://tools.ietf.org/html/rfc6749>) from the original on 15 October 2012. Retrieved 10 October 2012.

3. Whitson, Gordon. "Understanding OAuth: What Happens When You Log Into a Site with Google, X, or Facebook" (<https://lifehacker.com/5918086/understanding-oauth-what-happens-when-you-log-into-a-site-with-google-twitter-or-facebook>). *Lifehacker*. Archived (<https://web.archive.org/web/20140424052409/http://lifehacker.com/5918086/understanding-oauth-what-happens-when-you-log-into-a-site-with-google-twitter-or-facebook>) from the original on 24 April 2014. Retrieved 15 May 2016.
4. Henry, Gavin (January 2020). "Justin Richer on OAuth" (<https://doi.org/10.1109%2FMS.2019.2949648>). *IEEE Software*. **37** (1): 98–100. doi:10.1109/MS.2019.2949648 (<https://doi.org/10.1109%2FMS.2019.2949648>). ISSN 0740-7459 (<https://search.worldcat.org/issn/0740-7459>).
5. "Amazon & OAuth 2.0" (<https://login.amazon.com/>). Archived (<https://web.archive.org/web/20171208010412/https://login.amazon.com/>) from the original on 8 December 2017. Retrieved 15 December 2017.
6. "OAuth Introduction" (<https://oauth.net/about/introduction/>). Archived (<https://web.archive.org/web/20260401225859/https://oauth.net/about/introduction/>) from the original on 1 April 2026. Retrieved 1 April 2026.
7. Chris Crum (31 August 2010). "Twitter Apps Go OAuth Today" (<http://www.webpronews.com/twitter-apps-go-oauth-today-2010-08/>). *WebProNews*. Archived (<https://web.archive.org/web/20170731164856/http://www.webpronews.com/twitter-apps-go-oauth-today-2010-08/>) from the original on 31 July 2017. Retrieved 31 July 2017.
8. Jones, Michael; Hardt, Dick (October 2012). "RFC6750 - The OAuth 2.0 Authorization Framework: Bearer Token Usage" (<https://tools.ietf.org/html/rfc6750>). Internet Engineering Task Force. doi:10.17487/RFC6750 (<https://doi.org/10.17487%2FRFC6750>). Archived (<https://web.archive.org/web/20121015232219/http://tools.ietf.org/html/rfc6750>) from the original on 15 October 2012. Retrieved 10 October 2012.
9. Lodderstedt, Torsten; Hardt, Dick; Parecki, Aaron (15 November 2024). "The OAuth 2.1 Authorization Framework" (<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-12>). tools.ietf.org. Retrieved 19 December 2024.
10. "OAuth Security Advisory: 2009.1" (<http://oauth.net/advisories/2009-1>). *OAuth Community Site*. 23 April 2009. Archived (<https://web.archive.org/web/20160527002938/http://oauth.net/advisories/2009-1/>) from the original on 27 May 2016. Retrieved 23 April 2009.
11. "OAuth Core 1.0a" (<http://oauth.net/core/1.0a>). *OAuth Community Site*. Archived (<https://web.archive.org/web/20090630092220/http://oauth.net/core/1.0a>) from the original on 30 June 2009. Retrieved 17 July 2009.
12. Lodderstedt, Torsten; McGloin, Mark; Hunt, Phil (January 2013). Lodderstedt, T (ed.). "RFC6819 - OAuth 2.0 Threat Model and Security Considerations" (<https://tools.ietf.org/html/rfc6819.html>). *Internet Engineering Task Force*. doi:10.17487/RFC6819 (<https://doi.org/10.17487%2FRFC6819>). Archived (<https://web.archive.org/web/20200630000422/https://tools.ietf.org/html/rfc6819>) from the original on 30 June 2020. Retrieved 29 June 2020.[rfc:6819 OAuth 2.0 Threat Model and Security Considerations]. Internet Engineering Task Force. Accessed January 2015.
13. "OAuth Security Advisory: 2014.1 "Covert Redirect" " (<http://oauth.net/advisories/2014-1-covert-redirect/>). *OAuth Community Site*. 4 May 2014. Archived (<https://web.archive.org/web/20151121111134/http://oauth.net/advisories/2014-1-covert-redirect/>) from the original on 21 November 2015. Retrieved 10 November 2014.
14. "Serious security flaw in OAuth, OpenID discovered" (<https://www.cnet.com/news/serious-security-flaw-in-oauth-and-openid-discovered/>). *CNET*. 2 May 2014. Archived (<https://web.archive.org/web/20151102002904/http://www.cnet.com/news/serious-security-flaw-in-oauth-and-openid-discovered/>) from the original on 2 November 2015. Retrieved 10 November 2014.

15. "Math student detects OAuth, OpenID security vulnerability" (<http://phys.org/news/2014-05-math-student-oauth-openid-vulnerability.html>). Phys.org. 3 May 2014. Archived (<https://web.archive.org/web/20151106024436/http://phys.org/news/2014-05-math-student-oauth-openid-vulnerability.html>) from the original on 6 November 2015. Retrieved 11 November 2014.
16. "Covert Redirect" (http://tetraph.com/covert_redirect/). Tetraph. 1 May 2014. Archived (http://web.archive.org/web/20160310005903/http://tetraph.com/covert_redirect/) from the original on 10 March 2016. Retrieved 10 November 2014.
17. Fett, Daniel; Küsters, Ralf; Schmitz, Guido (2016). "A Comprehensive Formal Security Analysis of OAuth 2.0". *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. New York, New York, USA: ACM Press. pp. 1204–1215. arXiv:1601.01229 (<https://arxiv.org/abs/1601.01229>). Bibcode:2016arXiv160101229F (<http://ui.adsabs.harvard.edu/abs/2016arXiv160101229F>). doi:10.1145/2976749.2978385 (<http://doi.org/10.1145/2976749.2978385>). ISBN 9781450341394. S2CID 1723789 (<https://api.semanticscholar.org/CorpusID:1723789>).
18. Bradley, John; Labunets, Andrey; Lodderstedt, Torsten; Fett, Daniel (8 July 2019). "OAuth 2.0 Security Best Current Practice" (<https://tools.ietf.org/html/draft-ietf-oauth-security-topics-13.html>). *Internet Engineering Task Force*. Archived (<https://web.archive.org/web/20200117063230/https://tools.ietf.org/html/draft-ietf-oauth-security-topics-13.html>) from the original on 17 January 2020. Retrieved 29 July 2019.
19. "Hacking Facebook with OAuth 2.0 and Chrome" (<http://homakov.blogspot.co.uk/2013/02/hacking-facebook-with-oauth2-and-chrome.html>). 12 February 2013. Archived (<https://web.archive.org/web/20160423083324/http://homakov.blogspot.co.uk/2013/02/hacking-facebook-with-oauth2-and-chrome.html>) from the original on 23 April 2016. Retrieved 6 March 2013.
20. "Google Docs phishing email 'cost Minnesota \$90,000' " (<https://www.bbc.co.uk/news/technology-39845545>). *BBC News*. 8 May 2017. Archived (<https://web.archive.org/web/20200630000812/https://www.bbc.com/news/technology-39845545>) from the original on 30 June 2020. Retrieved 29 June 2020.
21. "OAuth Grant Types" (<https://oauth.net/2/grant-types/>). *OAuth.net*. Retrieved 6 December 2023.
22. "Authentication - Facebook Developers" (<https://developers.facebook.com/docs/authentication/>). *Facebook for Developers*. Archived (<https://web.archive.org/web/20140123045312/http://developers.facebook.com/docs/authentication/>) from the original on 23 January 2014. Retrieved 5 January 2020.
23. "Using OAuth 2.0 to Access Google APIs | Google Identity Platform" (<https://developers.google.com/identity/protocols/OAuth2>). *Google Developers*. Archived (<https://web.archive.org/web/20200104215722/https://developers.google.com/identity/protocols/OAuth2>) from the original on 4 January 2020. Retrieved 4 January 2020.
24. "v2.0 Protocols - OAuth 2.0 Authorization Code Flow" (<https://docs.microsoft.com/en-us/azure/active-directory/active-directory-v2-protocols-oauth-code>). *Microsoft Docs*. Archived (<https://web.archive.org/web/20200629235721/https://docs.microsoft.com/en-us/azure/active-directory/develop/v2-oauth2-auth-code-flow>) from the original on 29 June 2020. Retrieved 29 June 2020.
25. "An Introduction to OAuth2 Authentication" (<https://www.linode.com/docs/guides/oauth2-authentication-an-introduction/>). *Akamai Connected Cloud*. 22 October 2021. Archived (<https://web.archive.org/web/20240418113137/https://www.linode.com/docs/guides/oauth2-authentication-an-introduction/>) from the original on 18 April 2024. Retrieved 18 April 2024.
26. "End User Authentication with OAuth 2.0" (<http://oauth.net/articles/authentication/>). *OAuth Community Site*. Archived (<https://web.archive.org/web/20151119133521/http://oauth.net/articles/authentication/>) from the original on 19 November 2015. Retrieved 8 March 2016.

27. Hammer, Eran (28 July 2012). "OAuth 2.0 and the Road to Hell" (<https://web.archive.org/web/20130325140509/http://hueniverse.com/2012/07/oauth-2-0-and-the-road-to-hell/>). *Hueniverse*. Archived from the original (<https://hueniverse.com/oauth-2-0-and-the-road-to-hell-8eec45921529>) on 25 March 2013. Retrieved 17 January 2018.
28. Harris, David (October 2021). "October 2021 - Updates and reassurances" (https://www.pmail.com/devnews_archive.htm). *Pegasus Mail*. Archived (https://web.archive.org/web/20240116142819/https://www.pmail.com/devnews_archive.htm) from the original on 16 January 2024. Retrieved 19 December 2024.

External links

- Hardt, Dick (October 2012). "The OAuth 2.0 Authorization Framework" (<https://datatracker.ietf.org/doc/html/rfc6749>). Internet Engineering Task Force.
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=OAuth&oldid=1354517237>"